16_02_2021

Tuesday, February 16, 2021 12:04 AM

Announcements

Exam 1 on Thursday – please take practice quiz

Project 1 verification out on Canvas

Exam coverage: Lecture until 02/09, HWs 1 & 2, Project 1

So Far

How caches work?

Techniques to reduce miss penalty → victim $, critical word first, etc

Today

MR reduction – compiler optimizations (Miss rate)

Layout for Data via Merging

- Lay out arrays that are accessed together in the same array (i.e., interleaved in memory)

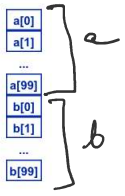
Code:

```
int a[100];
int b[100];

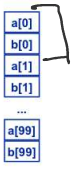
for (i = 0; i < MAX; i += 2)
   a[i] = b[i];
```

0, 2, 4,

Unoptimized:



Compiler optimized:



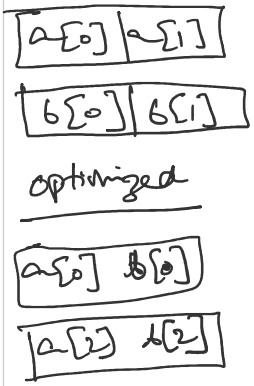
What if block size = two "int"s? Each access to a[i] or b[i] brings in a[i+1] and b[i+1], which are never used

Now every access to a missing block brings in useful data as well (we've enhanced the spatial locality of the code).

Problem: DANGEROUS because some programmers assume a specific layout! e.g., *(a+1) == a[1] in C, but now == b[0]

CACHE-56

block



Other optimizations (cont.):

- **Loop Fusion**
  - Two loops with identical sets of iterations can be combined:

```
for (j = 0; j < MAXJ; j++)
   for (i = 0; i < MAXI; i++)
     a[i][j] = cos(x[i][j]);
// other stuff
for (j = 0; j < MAXJ; j++)
   for (i = 0; i < MAXI; i++)
     z[i][j] = a[i][j] * d[i];
```

problem is that a[i][j] is written entirely in the first loop before it can be read in the second loop, reducing the *chance* that it is still in the cache (a guess on the part of the compiler)

```
for (j = 0; j < MAXJ; j++)
   for (i = 0; i < MAXI; i++) {
     a[i][j] = cos(x[i][j]);
     z[i][j] = a[i][j] * d[i];
   }
```

problem reduced by "fusing" the two loops

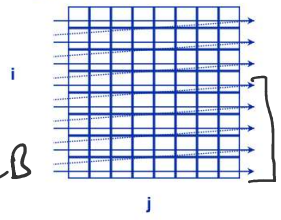
CACHE-57

Other optimizations (cont.):

- **Tiling (also called, confusingly, "blocking")**

  Idea: access "regions" of arrays instead of whole array

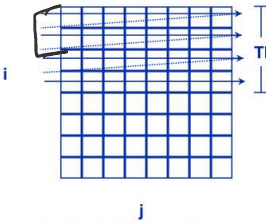
Each value of k:




```
for (k = 0; k < N; k++)
   for (i = 0; i < N; i++)
      for (j = 0; j < N; j++)
         sum += x[i][j]*(k+1);
```

problem: *not all of x[i][j] can fit in the cache*

for (k = 0; k < N; k++)




```
for (ii = 0; ii < N; ii += TF)
   for (k = 0; k < N; k++)
      for (i = ii; i < ii+TF; i++)
         for (j = 0; j < N; j++)
            sum += x[i][j]*(k+1);
```

Tiling factors "TF" selected so that a "tile" of x[i][j] fits in the cache

Now only compulsory misses for x[i][j]

CACHE-58

Other optimizations (cont.):

- More about tiling

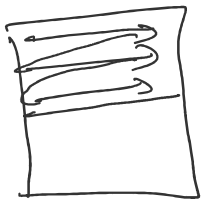
Why not tile in both i and j directions?

Breaks vectorization (if compiler vectorizes)

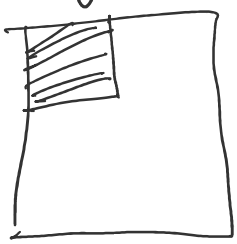
```
for (ii = 0; ii < N; ii += TF)
    for (k = 0; k < N; k++)
      for (i = ii; i < ii+TF; i++)
        for (j = 0; j < N; j++)
            sum += x[i][j]*(k+1);
```

Usually "vectorize" (use SIMD packed instructions) for inner most loop

CACHE-59



Why not



Example of $ Tiling

```
for (k=0; k < N; k++)
    for (i=0; i<N; i++)
        for (j=0; j<N; j++)
            x[i][j] = x[i][j] * y[k]
```

(C=10, B=4, S=6)

N = 32, size of int = 4B

Each block = 16B ⇒ store 4 integers

of blocks = $2^{(C-B)} = 2^6 = 64$ blocks

Assume only x stored in $.

How many misses if $ initially empty?

Access x[0][0]

fetch x[0][0], x[0][1] .. x[0][3]

untiled misses

$32 \times 32 \times 8$ misses

256 entries of X

Now lets examine the tiled version

```
for (ii=0; ii < N; ii+=TF)
    for (k=0; k < N; k++)
        for (i=ii; i< ii+TF; i++)
            for (j=0; j< N; j++)
                x[i][j] = x[i][j] + y[k]
```

original kernel

TF = 8
And row-major
i.e x[i][j] is next to x[i][j+1]

each block = 4 ints
each line = 32 ints
fit 8 lines in $

How many misses occur now?

$$\left[4 \times \frac{8 \times 8}{8 \text{ rows fetched for 1 iteration of k loop}}\right] \text{ misses in inner most loop}$$

Other optimizations:

- Loop interchange
  - Increase temporal locality by exchanging inner and outer loops

for k = 0 ... N
x[i][j] * k

```
for (k = 0; k < N; k++)
  for (i = 0; i < N; i++)
    for (j = 0; j < N; j++)
      sum += x[i][j]*(k+1);
```

⇩

```
for (i = 0; i < N; i += 1)
  for (j = 0; j < N; j += 1)
    for (k = 0; k < N; k++)
      sum += x[i][j]*(k+1);
```

Note: cannot vectorize inner most loop now

This is why tiling is used instead of loop interchange is

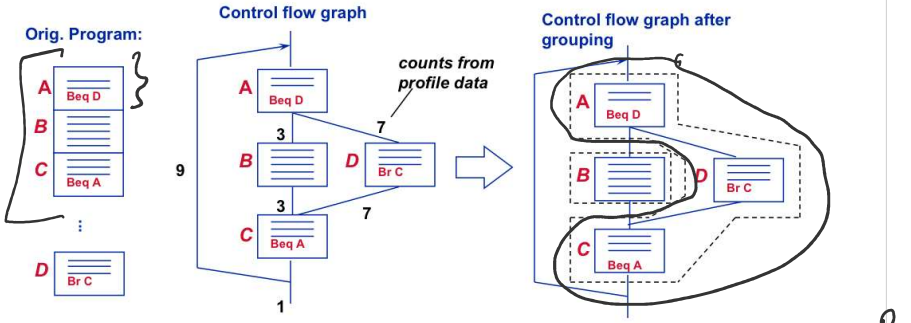
CACHE-60

Compiler layout of inst./data

- Idea: if fetch A then B, put A & B in the same block (spatial locality)
- For instruction caches:
  - For most instructions, this happens normally
  - Branches change the sequential access pattern
  - Solution:
    - Figure out how frequently every branch is taken/not-taken
    - Form control flow graph
      - Draw boxes around largest sequential runs of code (see example on next page)
    - Find groups of instructions that tend to execute one after another
    - Rewrite program putting these instructions into sequential order

CACHE-61

Instruction layout



A D C B

- Write out program:
  - Instructions in this order: A, D, C, then B

CACHE-62

FAST
SMALL
EXPENSIVE
←
Memory
→
LARGE
SLOW
CHEAP